МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙГОСУДАРСТВЕННЫЙТЕХНИЧЕСКИЙУНИВЕРСИТЕТ» ФАКУЛЬТЕТ

ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил:
А.С. Мышкевич
студент группы ПО-13
Проверил:
А.А. Крощенко
доц. кафедры ИИТ,
19.03.2026

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

6. Анализ вклада разработчика в open-source на GitHub

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя имя пользователя GitHub.
2. Получает все публичные репозитории, в которые этот пользователь вносил вклад (не только его собственные, но и форки, pull requests).
3. Для каждого репозитория определяет:

 Количество коммитов пользователя

 Количество открытых pull requests

 Количество закрытых pull requests

 Количество issues, созданных пользователем

4. Вычисляет уровень активности пользователя, например, на основе взвешенной формулы:

Активность = (Коммиты * 1) + (Открытые PR * 2) + (Закрытые PR * 3) + (Issues * 1.5)

5. Определяет самый активный проект, в котором пользователь работал.
6. Сохраняет данные в github_contribution.json.

Код:

```
import os
from datetime import datetime
import json
import pandas as pd
from github import Github
from dotenv import load_dotenv

# Авторизация через Personal Access Token (PAT)
load_dotenv()

token = os.getenv("GITHUB_TOKEN")
if not token:
    print("Ошибка: Не найден GITHUB_TOKEN в файле .env")
    print("Создайте файл .env с содержимым: GITHUB_TOKEN=ваш_токен")
    exit(1)

g = Github(token)

# Запрос имени пользователя
username = input("Введите имя пользователя GitHub: ")
print(f"\nАнализируем вклад пользователя '{username}'...")
```

```
try:
    # Получаем пользователя
    user = g.get_user(username)

    # Множество для хранения уникальных репозиториев (чтобы избежать дубликатов)
    repos_dict = {}

    # 1. Получаем репозитории, созданные пользователем
    print("Получаем репозитории пользователя...")
    for repo in user.get_repos():
        repos_dict[repo.full_name] = {
            "full_name": repo.full_name,
            "owner": repo.owner.login,
            "name": repo.name
        }

    # 2. Получаем репозитории, которые пользователь форкнул
    print("Получаем форкнутые репозитории...")
    for repo in user.get_repos():
        if repo.fork:
            repos_dict[repo.full_name] = {
                "full_name": repo.full_name,
                "owner": repo.owner.login,
                "name": repo.name
            }

    # 3. Получаем репозитории через pull requests
    print("Получаем pull requests пользователя...")
    query = f"is:pr author:{username} is:merged"
    for pr in g.search_issues(query):
        repo_name = pr.repository.full_name
        if repo_name not in repos_dict:
            repos_dict[repo_name] = {
                "full_name": repo_name,
                "owner": pr.repository.owner.login,
                "name": pr.repository.name
            }

    # 4. Получаем репозитории через issues
    print("Получаем issues пользователя...")
    query = f"is:issue author:{username}"
    for issue in g.search_issues(query):
        repo_name = issue.repository.full_name
        if repo_name not in repos_dict:
            repos_dict[repo_name] = {
                "full_name": repo_name,
                "owner": issue.repository.owner.login,
                "name": issue.repository.name
            }
```

```
print(f"\nНайдено репозиториев: {len(repos_dict)}")

# Собираем данные по каждому репозиторию
contributions = []

for repo_name, repo_info in repos_dict.items():
    print(f"Анализируем: {repo_name}")

    try:
        repo = g.get_repo(repo_name)

        # Считаем коммиты пользователя в репозитории
        commits_count = 0
        try:
            # Получаем коммиты от пользователя
            commits = repo.get_commits(author=user)
            commits_count = sum(1 for _ in commits)
        except Exception as e:
            print(f"  Не удалось получить коммиты: {e}")

        # Считаем pull requests пользователя
        open_prs_count = 0
        closed_prs_count = 0

        # Открытые PR
        try:
            open_prs = repo.get_pulls(state='open', sort='created')
            for pr in open_prs:
                if pr.user.login == username:
                    open_prs_count += 1
        except Exception as e:
            print(f"  Не удалось получить открытые PR: {e}")

        # Закрытые PR (включая объединенные и закрытые без слияния)
        try:
            closed_prs = repo.get_pulls(state='closed', sort='created')
            for pr in closed_prs:
                if pr.user.login == username:
                    closed_prs_count += 1
        except Exception as e:
            print(f"  Не удалось получить закрытые PR: {e}")

        # Считаем issues, созданные пользователем
        issues_count = 0
        try:
            issues = repo.get_issues(state='all', creator=user)
            issues_count = sum(1 for _ in issues)
        except Exception as e:
            print(f"  Не удалось получить issues: {e}")
```

```
            # Вычисляем активность по формуле
            activity_score = (commits_count * 1) + (open_prs_count * 2) + (closed_prs_count *
3) + (issues_count * 1.5)

            if commits_count > 0 or open_prs_count > 0 or closed_prs_count > 0 or
issues_count > 0:
                contributions.append({
                    "repository": repo_name,
                    "commits": commits_count,
                    "open_pull_requests": open_prs_count,
                    "closed_pull_requests": closed_prs_count,
                    "created_issues": issues_count,
                    "activity_score": activity_score
                })

        except Exception as e:
            print(f"  Ошибка при анализе {repo_name}: {e}")

    # Создаем DataFrame для удобного анализа
    df = pd.DataFrame(contributions)

    if df.empty:
        print(f"\nПользователь {username} не имеет видимой активности в публичных
репозиториях.")
    else:
        # Общая статистика
        total_commits = df['commits'].sum()
        total_open_prs = df['open_pull_requests'].sum()
        total_closed_prs = df['closed_pull_requests'].sum()
        total_issues = df['created_issues'].sum()
        total_activity = df['activity_score'].sum()

        # Самый активный проект
        most_active = df.loc[df['activity_score'].idxmax()]

        # Вывод результатов в консоль
        print(f"\n{'='*50}")
        print(f"Результаты анализа пользователя: {username}")
        print(f"{'='*50}")
        print(f"Всего репозиториев с активностью: {len(df)}")
        print(f"\nОбщая статистика:")
        print(f"  - Общее количество коммитов: {total_commits}")
        print(f"  - Открытых pull requests: {total_open_prs}")
        print(f"  - Закрытых pull requests: {total_closed_prs}")
        print(f"  - Созданных issues: {total_issues}")
        print(f"  - Общая активность: {total_activity:.1f} баллов")

        print(f"\nСамый активный проект:")
        print(f"  - {most_active['repository']}")
        print(f"  - Коммитов: {most_active['commits']}")
```

```
        print(f"  - Открытых PR: {most_active['open_pull_requests']}")
        print(f"  - Закрытых PR: {most_active['closed_pull_requests']}")
        print(f"  - Issues: {most_active['created_issues']}")
        print(f"  - Активность: {most_active['activity_score']:.1f} баллов")

        # Сохраняем в JSON
        output_data = {
            "username": username,
            "analysis_date": datetime.now().isoformat(),
            "total_repositories": len(df),
            "total_commits": int(total_commits),
            "total_open_pull_requests": int(total_open_prs),
            "total_closed_pull_requests": int(total_closed_prs),
            "total_created_issues": int(total_issues),
            "total_activity_score": float(total_activity),
            "most_active_project": {
                "repository": most_active['repository'],
                "commits": int(most_active['commits']),
                "open_pull_requests": int(most_active['open_pull_requests']),
                "closed_pull_requests": int(most_active['closed_pull_requests']),
                "created_issues": int(most_active['created_issues']),
                "activity_score": float(most_active['activity_score'])
            },
            "all_contributions": contributions
        }

        # Сохраняем в файл
        with open('github_contribution.json', 'w', encoding='utf-8') as f:
            json.dump(output_data, f, ensure_ascii=False, indent=2)

        print(f"\nРезультаты сохранены в 'github_contribution.json'")

        # Дополнительно: топ-5 проектов по активности
        print(f"\nТоп-5 проектов по активности:")
        top5 = df.nlargest(5, 'activity_score')[['repository', 'commits', 'activity_score']]
        for idx, row in top5.iterrows():
            print(f"  - {row['repository']}: {row['activity_score']:.1f} баллов
({row['commits']} коммитов)")

except Exception as e:
    print(f"Произошла ошибка: {e}")
    print("Проверьте имя пользователя и наличие доступа к API GitHub")
```

```
Всего репозиториев с активностью: 9

Общая статистика:
  - Общее количество коммитов: 24
  - Открытых pull requests: 0
  - Закрытых pull requests: 0
  - Созданных issues: 3
  - Общая активность: 28.5 баллов
```

Вывод: были отработаны базовые знания GitHubApi на python при решении практических задач.